# ioNova AI Lab

## App User Guide

Address Structuring Lab (App A) and isolation test app (App B) on Microsoft Fabric, using the existing Microsoft Foundry address-structuring agent

| | |
|---|---|
| **Version** | 1.0 |
| **Date** | 2 November 2026 |
| **Prepared by** | Vilas Menon |
| **Environment** | Development (Microsoft Fabric trial, Central India) |
| **Classification** | Internal |

# Version history

## Document control

| | |
|---|---|
| **Document** | ioNova AI Lab: App User Guide |
| **Current version** | 1.0 |
| **Status** | Released for internal use |
| **Owner** | Vilas Menon |
| **Applies to** | App A (Address Structuring Lab) and App B (isolation test app), ioNova dev environment, repository main branch as of 2 November 2026 |
| **Source repository** | github.com/vilasvmenon-ai/azure-fabric-proj (docs/user-guide) |

## Revision history

| Version | Date | Author | Changes |
|---|---|---|---|
| 1.0 | 2 November 2026 | Vilas Menon | First release. Step-by-step guide to both apps with screenshots from the dev environment. |

## Review and approval

| Name | Role | Signature | Date |
|---|---|---|---|
| Vilas Menon | Author | | 2 November 2026 |
| | Reviewer | | |
| | Approver | | |

# Index

**About this document** ..... **4**
Purpose ..... 4
Audience ..... 4
Scope ..... 4
How this guide is organised ..... 4
Conventions ..... 4
Related documents ..... 5
**1. The two apps at a glance** ..... **6**
**2. Open and sign in** ..... **7**
**3. Roles** ..... **8**
**4. First-time setup (app admin)** ..... **9**
**5. Structure an address** ..... **11**
**6. Records and record details** ..... **14**
**7. Review a result (reviewer)** ..... **16**
**8. Audit trail (auditor)** ..... **17**
**9. Administration (app admin)** ..... **18**
**10. App B, the isolation test app** ..... **19**

# About this document

## Purpose

This guide walks through both ioNova Fabric apps screen by screen, so that anyone who is new to them can sign in, structure an address, review results, check the audit trail and manage access. **App A**, the Address Structuring Lab, structures postal addresses with the existing Microsoft Foundry agent. **App B** is a small test app that proves each app stays separate.

## Audience

- **App users** who submit addresses and check their own records.
- **Reviewers** who approve or correct the agent's results.
- **Auditors** who read and verify the audit trail.
- **App administrators** who set up a deployment and grant roles.
- **Stakeholders** who want to understand what the apps do.

## Scope

The guide covers using the two apps in the ioNova **development** environment. It doesn't cover installing or deploying the apps, which is described in the setup and deployment guides in the repository (see Related documents).

## How this guide is organised

| Chapters | Read if you want to |
|---|---|
| 1 to 3 | Understand the two apps, sign in, and see which role gives which screens. |
| 4 | Set up a new deployment for the first time (app admin). |
| 5 and 6 | Structure an address and find your saved records (app user). |
| 7 | Review and correct results (reviewer). |
| 8 | Read and verify the audit trail (auditor). |
| 9 | Grant and revoke roles (app admin). |
| 10 | Use App B, the isolation test app. |

## Conventions

- **Bold** text is a button, tab, field or label exactly as it appears in the app.
- *Italic* text is a message the app shows.
- Numbered lists are steps to follow in order.
- Screenshots were taken in the dev environment on 2 November 2026 as vilas@ionova.ai, signed in as app admin and super user. All addresses shown are synthetic test data.

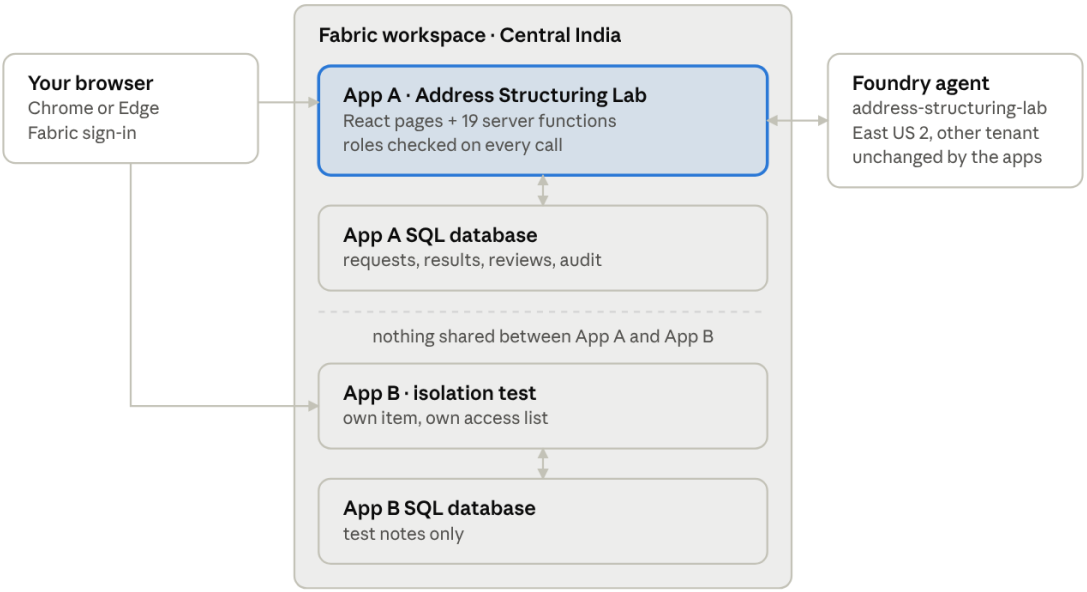
## Related documents

| Document | Where |
|---|---|
| ioNova Microsoft Fabric and Foundry React Apps PRD v1.2 | docs/prd/ in the repository |
| Setting up a new environment | docs/SETUP-NEW-ENVIRONMENT.md |
| Deploying with Claude Code | docs/CLAUDE-CODE-DEPLOY.md |
| Support, rollback and restore | docs/SUPPORT.md |
| Repository | github.com/vilasvmenon-ai/azure-fabric-proj |

## 1. The two apps at a glance

App A turns a free-text postal address into six checked fields using the existing Foundry agent; App B is a small second app that proves apps stay apart.

**App A calls the existing Foundry agent; App B shares nothing with it**




How the two apps fit together · dev environment

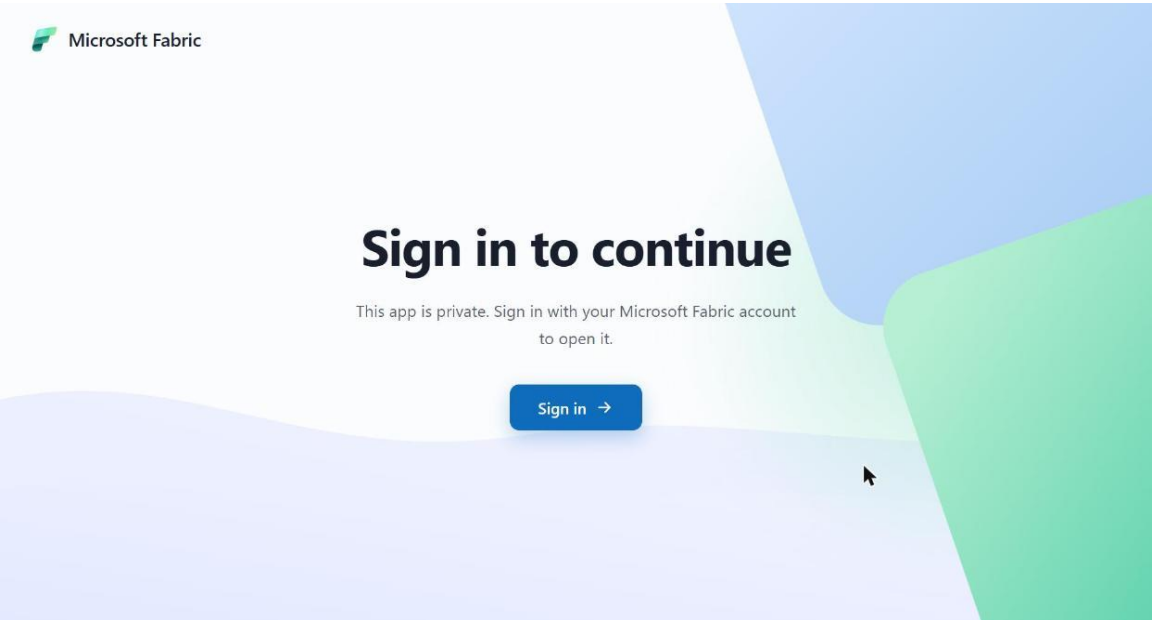
Only App A's server functions talk to the agent: they send the address text and save the answer in App A's own database. App B has its own item, database and access list, and nothing links it to App A.

## 2. Open and sign in

Both apps are private web pages hosted by Microsoft Fabric; you sign in with your ioNova work account.

| App | URL (dev) |
|---|---|
| App A, Address Structuring Lab | hot-shrub-288d05cfbd-centralindia.webapp.fabricapps.net |
| App B, isolation test | satin-gorge-cc665081ef-centralindia.webapp.fabricapps.net |

1. Open the app URL in Chrome or Edge.
2. Select **Sign in**. A Microsoft Fabric tab opens.
3. Pick your ioNova account (for example vilas@ionova.ai) and finish any MFA prompt. The Fabric tab closes and the app opens.
4. Your email and roles appear at the top right. **Sign out** ends the session.




A personal Microsoft account (gmail, outlook) cannot open the apps. You also need the app item shared with you in Fabric (**Run and interact**) and, for App A, a role granted inside the app.

## 3. Roles

What you see in App A depends on the roles an app admin granted you; one person can hold several roles, and nobody can change their own roles.

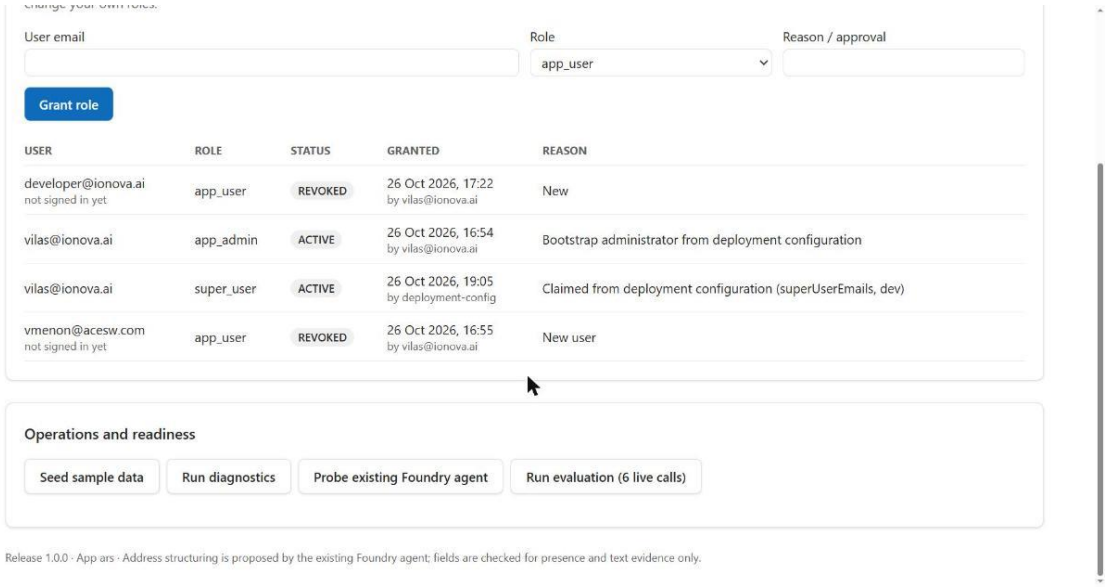
| Role | Can do | Tabs you see |
|---|---|---|
| app_user | Submit addresses, see and retry own requests | Structure an address, Records |
| reviewer | See all requests, approve, reject or correct fields | Records (all) |
| auditor | Read the audit trail and verify its integrity | Audit |
| audit_exporter | Read and export the audit trail | Audit |
| app_admin | Grant and revoke roles, run operations checks | Administration |
| super_user (dev/uat only) | app_user plus app_admin, so one person can check the system alone | Structure an address, Records, Administration |

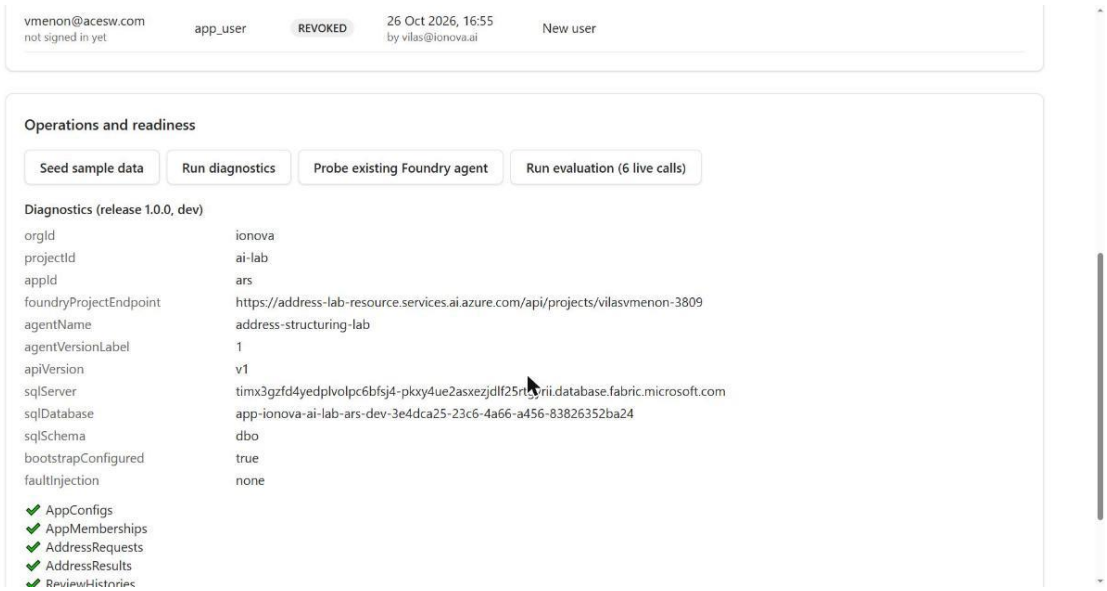
The screenshots in this guide were taken as vilas@ionova.ai with **app admin** and **super user**, so the Review and Audit screens are described but not shown. An app admin sees no address data or audit entries unless they also hold those roles.

# 4. First-time setup (app admin)

A new deployment needs one admin and a few readiness checks before anyone submits addresses; this was done in dev on 26 Oct 2026.

1. Sign in as an account listed in the deployment's bootstrap admins. On the **No access to this app yet** page, select **Become the first app administrator**.
2. Open **Administration**. Scroll to **Operations and readiness**.
3. Select **Seed sample data** once. It loads six approved synthetic addresses for the sample picker.
4. Select **Run diagnostics**. It lists the app's settings and a green tick for each database table.
5. Select **Probe existing Foundry agent**. Expect *reachable, contract valid*; it proves the app can call the agent.
6. Optional: **Run evaluation (6 live calls)** sends the six samples to the agent and compares the answers with the expected fields.



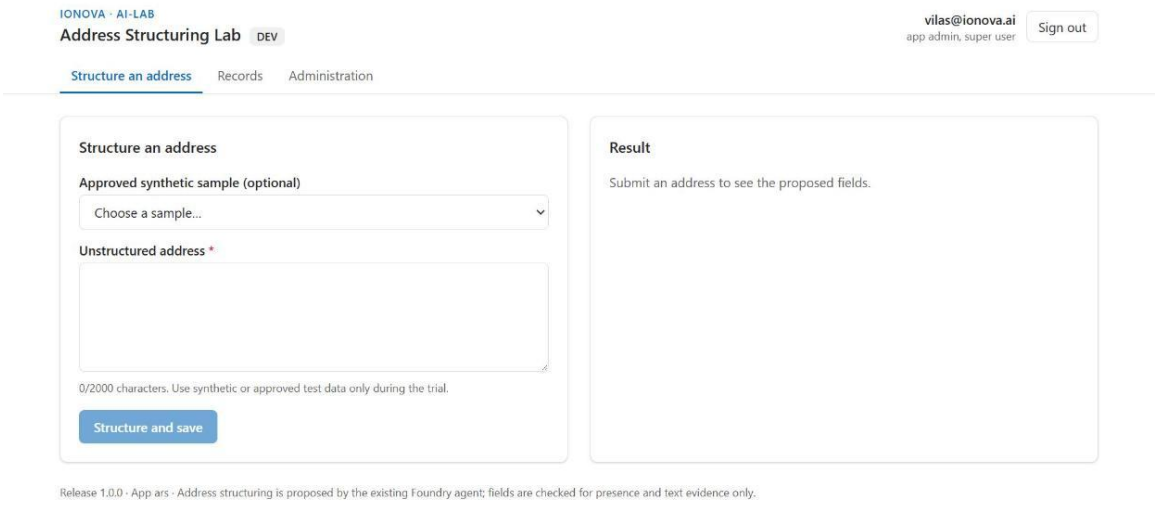



If the probe shows *auth (HTTP 403)*, the Foundry role is missing or still propagating (up to about 15 minutes).

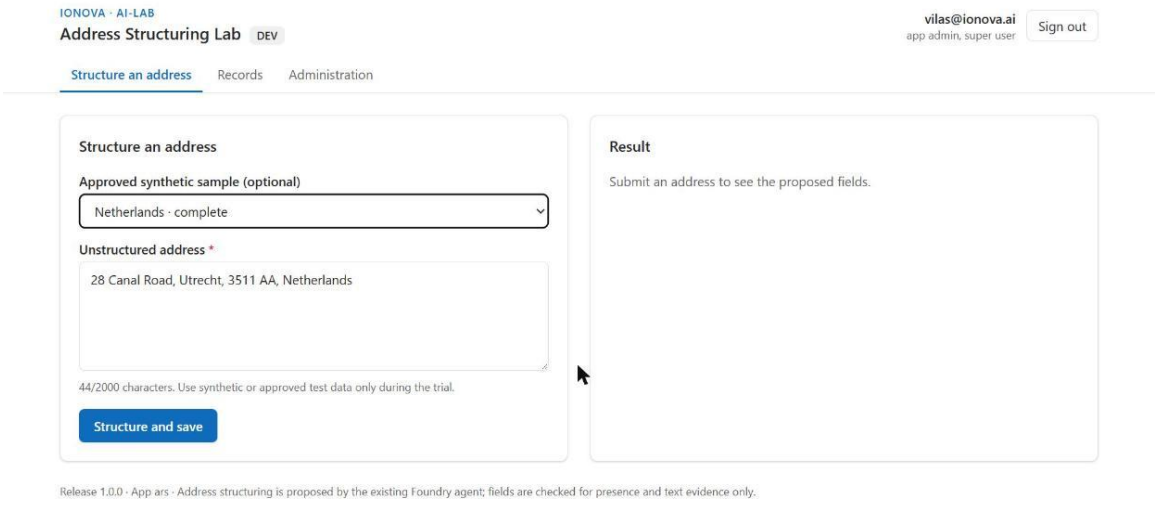
# 5. Structure an address

You paste one free-text address; the existing Foundry agent splits it into six fields, and the app saves both your input and the result (about 10–15 seconds).

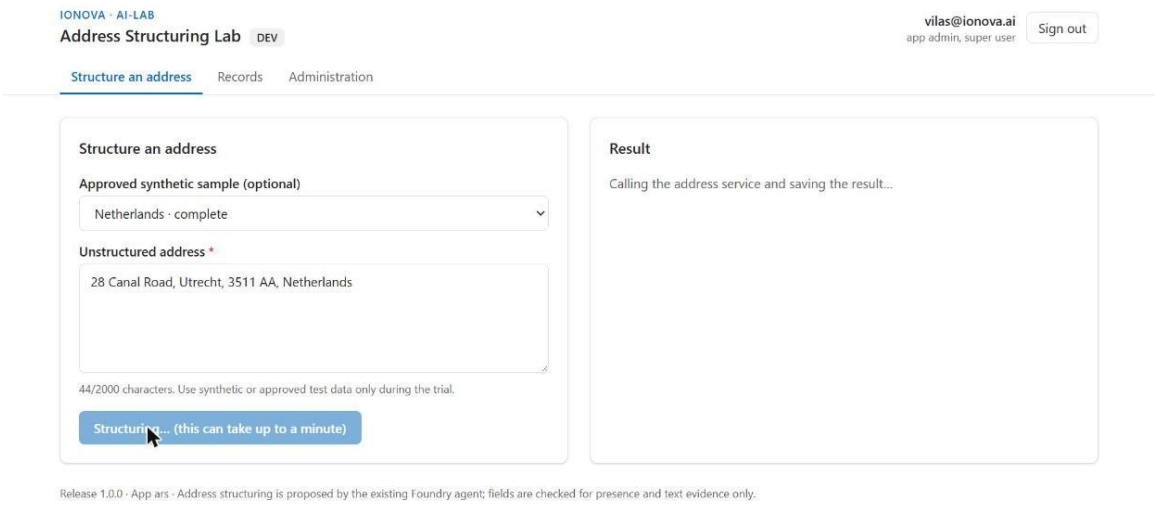
1. Open **Structure an address** (the first tab).



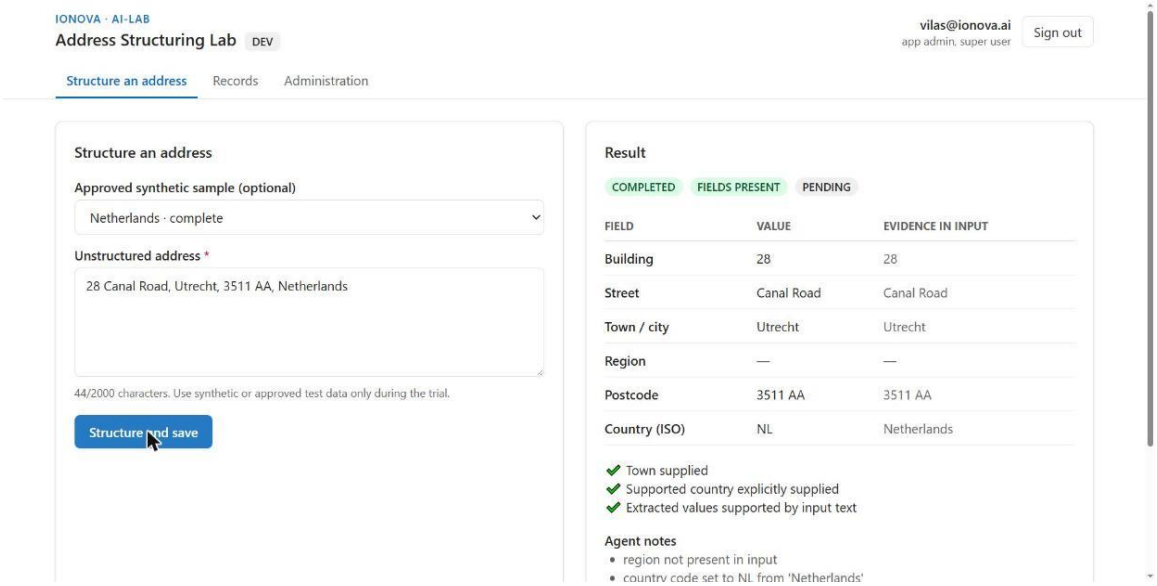

2. Either pick an entry from **Approved synthetic sample**, or type an address into **Unstructured address** (up to 2,000 characters). During the trial, use synthetic or approved test data only.



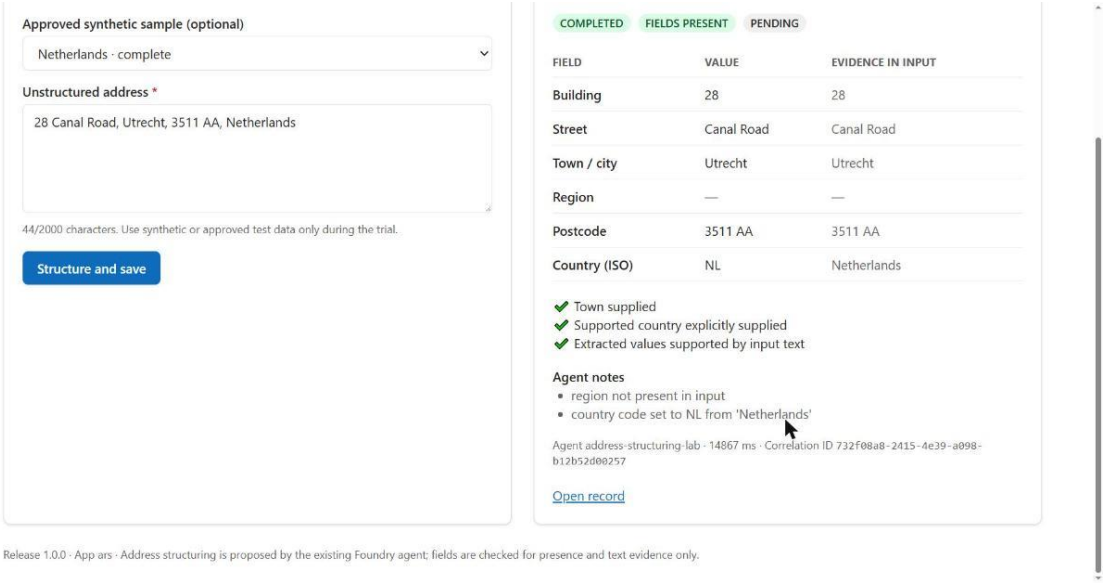

3. Select **Structure and save**. The button shows *Structuring… (this can take up to a minute)* while the agent works.




4. Read the **Result**: Building, Street, Town / city, Region, Postcode and Country (ISO), each with the words from your input that support it. A dash means the agent found nothing for that field.



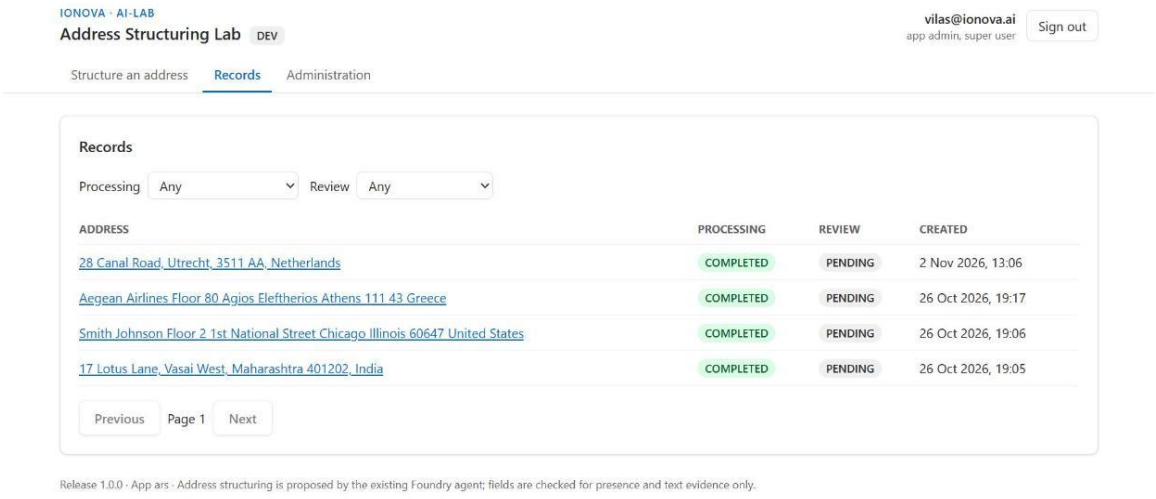

5. Check the badges and ticks. **COMPLETED** means it was saved, **FIELDS PRESENT** means the result passed the app's checks, and **PENDING** means no reviewer has looked at it yet. Below them are the agent's notes, the time taken and a **Correlation ID** for support.
6. Select **Open record** to see the saved record.


Approved synthetic sample (optional)
Netherlands · complete
Unstructured address *
28 Canal Road, Utrecht, 3511 AA, Netherlands
44/2000 characters. Use synthetic or approved test data only during the trial.
Structure and save
COMPLETED
FIELDS PRESENT
PENDING
FIELD
VALUE
EVIDENCE IN INPUT
Building
28
28
Street
Canal Road
Canal Road
Town / city
Utrecht
Utrecht
Region
—
—
Postcode
3511 AA
3511 AA
Country (ISO)
NL
Netherlands
Town supplied
Supported country explicitly supplied
Extracted values supported by input text
Agent notes
region not present in input
country code set to NL from 'Netherlands'
Agent address-structuring-lab · 14867 ms · Correlation ID 732f08a8-2415-4e39-a098-b12b52d00257
Open record
Release 1.0.0 · App ars · Address structuring is proposed by the existing Foundry agent; fields are checked for presence and text evidence only.

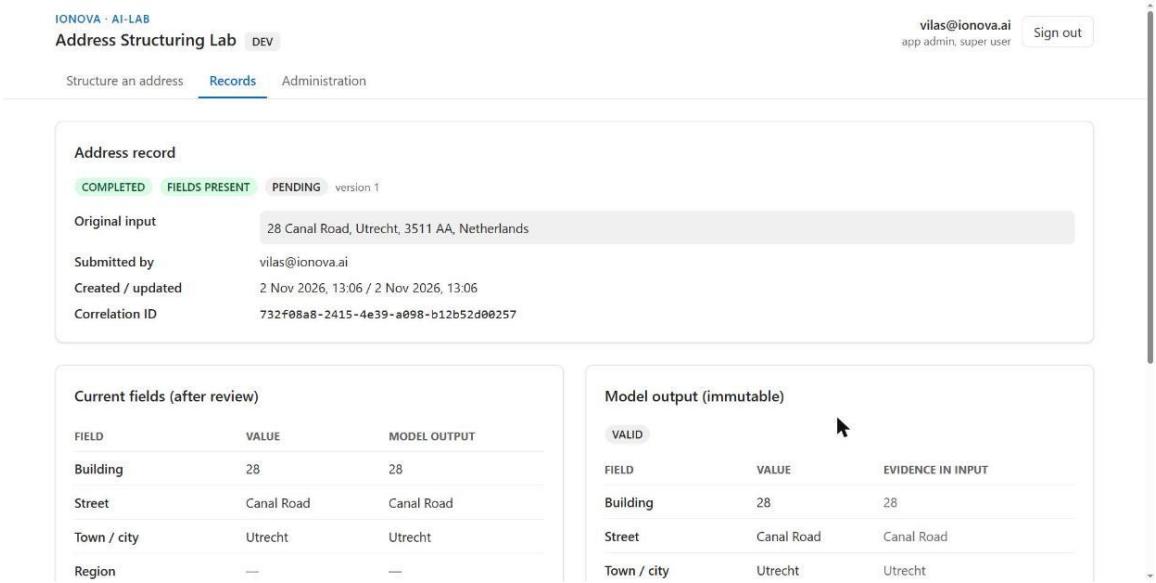
# 6. Records and record details

Every submission becomes a record that keeps your original input, the agent's untouched output and the current (reviewed) fields side by side.

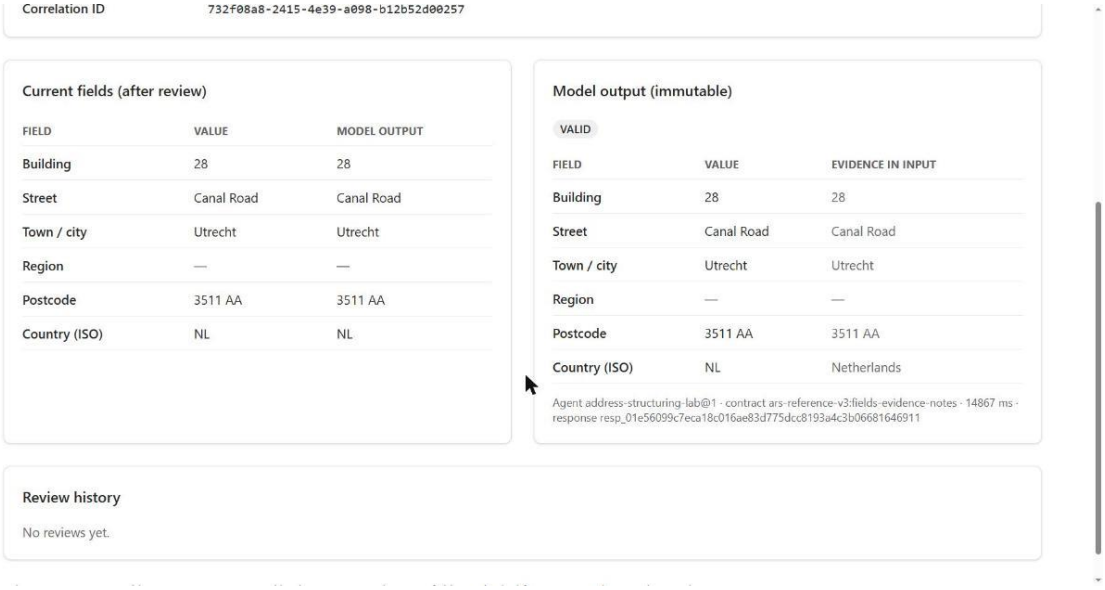
1. Open **Records**. Filter by **Processing** (for example COMPLETED or FAILED_AI) and **Review** (PENDING, APPROVED, REJECTED, NEEDS_CHANGES). App users see their own records; reviewers see everyone's.




2. Open a record (from the list, or **Open record** after a submission). The top card shows the status badges, version, original input, who submitted it, when, and the correlation ID.



3. Compare the two panels. **Current fields (after review)** is what the business uses; **Model output (immutable)** is the agent's answer, never edited, with the agent version, contract and response ID. **Review history** lists every reviewer decision.



## 7. Review a result (reviewer)

A reviewer confirms or corrects the agent's fields; the agent's original output is never changed, only the current fields and the review history.

1. Open **Records** and filter **Review** to **PENDING**.
2. Open a record. Below the panels shown above, a **Review** card appears for reviewers.
3. Under **Corrected fields (leave unchanged to keep)**, edit any of the six fields. Country takes a two-letter ISO code.
4. Choose a **Decision**: Approve, Needs changes, Reject or Comment only. Add **Comments** if useful.
5. Select **Save review (version N)**. The record's review badge and **Review history** update.
6. If someone else saved first, you see a conflict message. Select **Reload latest version** and review again.

No screenshot: the sample account does not hold the reviewer role.

## 8. Audit trail (auditor)

Every business action is written to a tamper-evident audit trail: each event carries a hash of the one before it, so any change or gap shows up when you verify it. Audit events never contain address text.

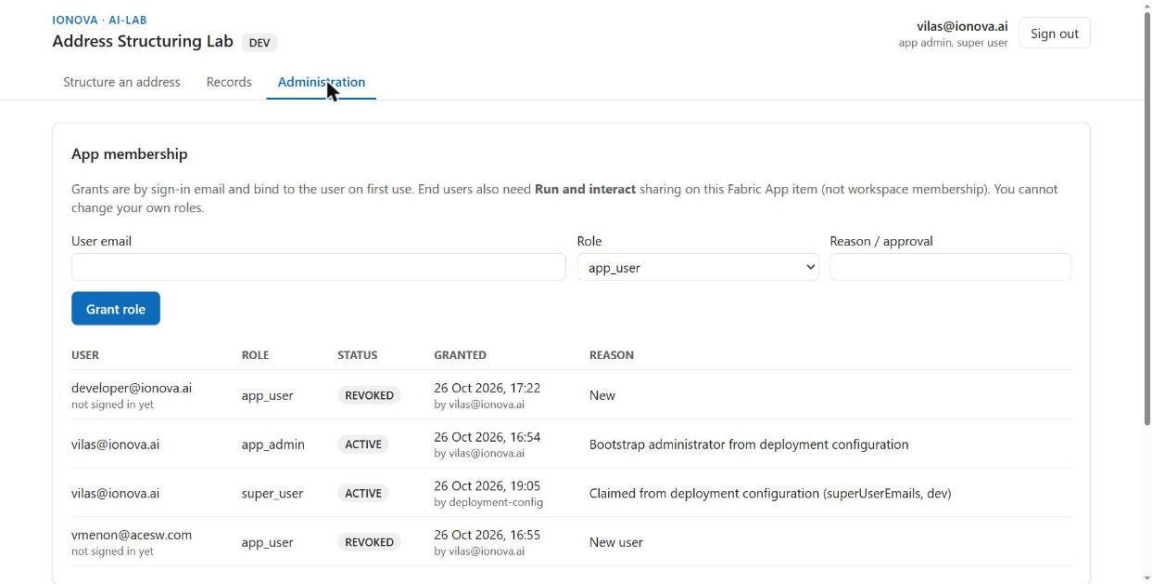
1. Open the **Audit** tab (shown only to auditor and audit_exporter).
2. Narrow the list with **Actor email**, **Action** (for example address.processed) and **Outcome** (SUCCESS, DENIED, FAILURE), then select **Apply filters**.
3. Read the table: **Time (local)**, **Actor**, **Action**, **Object**, **Outcome** and **Correlation ID**.
4. Select **Verify integrity**. Expect a green *Audit chain intact: N events verified*. A red *Audit chain BROKEN at event N* means something was altered; escalate to the project administrator.
5. With audit_exporter, **Export CSV** downloads the filtered events.

No screenshot: the sample account does not hold an audit role. In dev the chain was verified intact on 26 Oct 2026 (28 of 28 events).

# 9. Administration (app admin)

App admins decide who may use App A and in which role; every grant and revoke is audited with a reason.

1. Open **Administration**. The **App membership** card lists every grant with its role, status (ACTIVE or REVOKED), who granted it and why.
2. To grant a role, enter the person's sign-in **User email**, choose a **Role**, write a **Reason / approval**, then select **Grant role**. Repeat to give the same person a second role.
3. Make sure the person can also open the app: in Fabric, share the App A item with them as **Run and interact**. A role alone is not enough.
4. A new grant shows *not signed in yet* until the person's first visit, when it binds to their account.
5. You cannot change your own roles; another admin must do it.

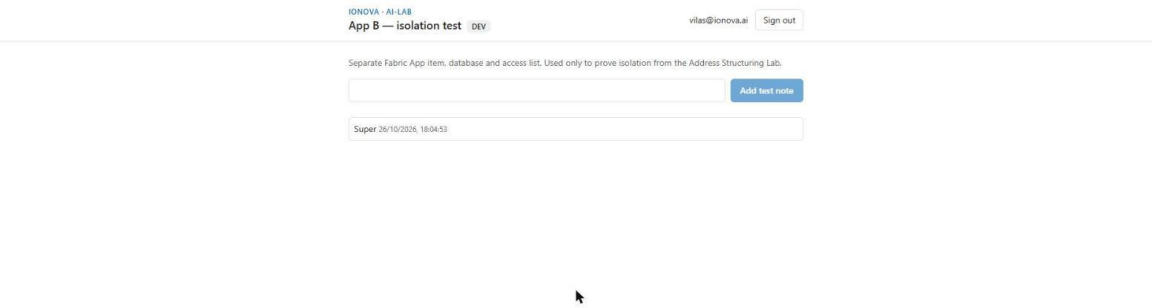



**Super user (dev and uat only).** An account listed in the deployment's super-user setting can select **Claim super user** on this tab once. It adds the super_user role (app_user plus app_admin) so one person can test end to end. Production refuses this setting.

## 10. App B, the isolation test app

App B is a deliberately tiny second app with its own Fabric item, database and access list; it exists to prove that one app cannot see another app's data or users.

1. Open the App B URL and sign in (a separate sign-in from App A).
2. Type a short note into the box and select **Add test note**.
3. Reload the page: the note is still listed with its date and time, so it was saved in App B's own database.




What it proved in dev (26 Oct 2026): App B's database holds only its notes table (1 note, “Super”), and App A's database has no notes table. Access is granted per app item, so a person shared on only one app cannot open the other.